

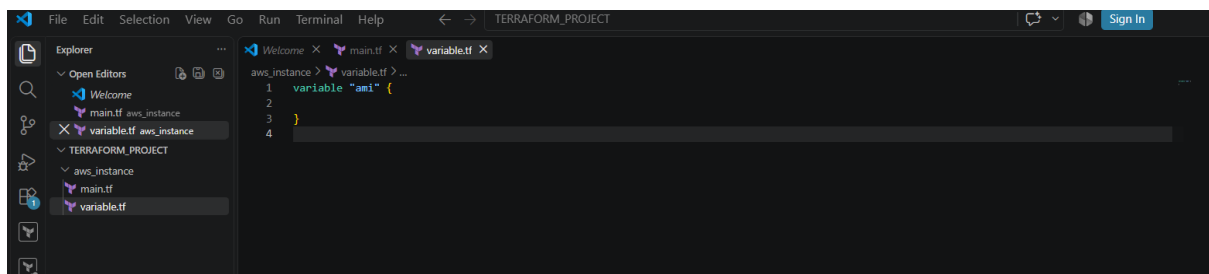
TERRAFORM – TASK – 06

2. Execute the script shown in the video.

- Open **VS Code** and create a project folder: **TERRAFORM_PROJECT**
- Inside it, create a folder named **aws_instance**
- Inside **aws_instance**, create these files:
 - main.tf
 - variables.tf
- In main.tf, write the EC2 resource block and use variable for AMI:
 - ami = var.ami
 - instance_type = "t3.micro"

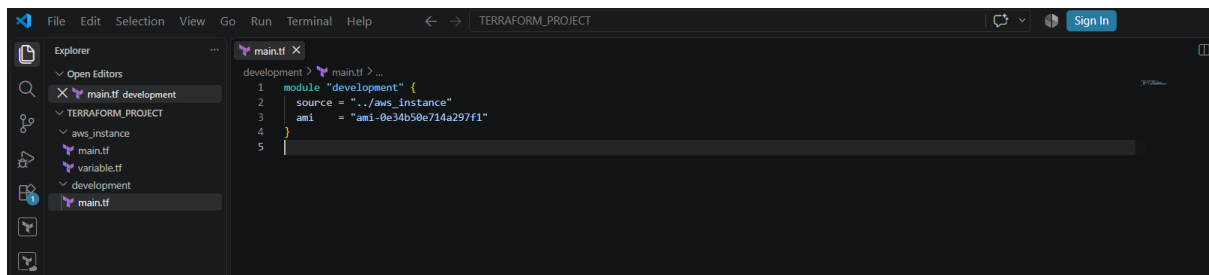


- Open aws_instance/variables.tf
- Create a variable for AMI like:
 - variable "ami" { }
- Save the file

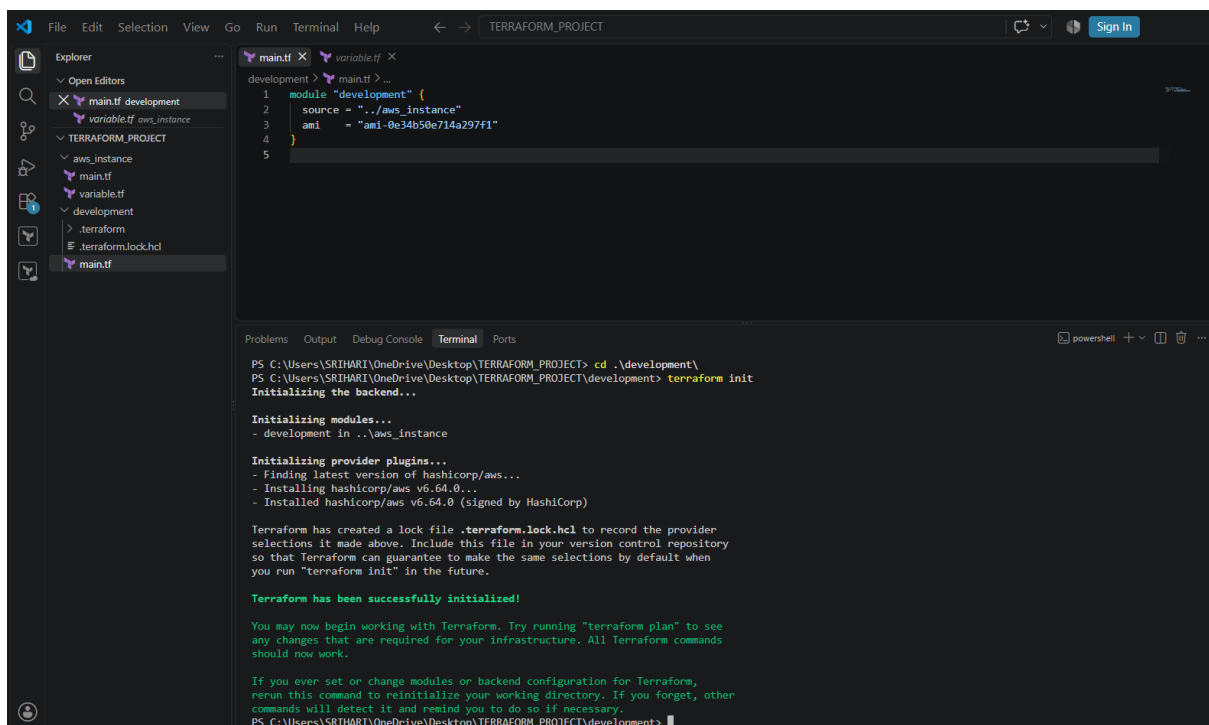


- Create a new folder in project: **development**
- Inside **development**, create main.tf

- In development/main.tf, call the module:
 - module "dev-webserver" {
 - source = "../aws_instance"
 - ami = "ami-xxxxxxxxxxxxx"
 - }



- Open terminal in VS Code
- Go to development folder:
- cd .\development\
- Run Terraform initialization:
- terraform init
- Confirm message: **Terraform has been successfully initialized**



- Run Terraform apply:
- terraform apply
- Review plan: **Plan: 1 to add, 0 to change, 0 to destroy**
- Type:
- yes
- Wait until it shows **Creation complete**
- Instance is successfully created

```

1  module "development" {
2    source = "../aws_instance"
3  }
4  ami = "ami-0e34b50e714a297f1"
5  }

Terraform will perform the following actions:

# module.development.aws_instance.my_instance will be created
+ resource "aws_instance" "my_instance" {
+   ami                  = "ami-0e34b50e714a297f1"
+   arn                  = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone     = (known after apply)
+   disable_api_stop      = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized         = (known after apply)
+   enable_primary_ipv6   = (known after apply)
+   force_destroy         = false
+   get_password_data     = false
+   host_id               = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile  = (known after apply)
+   id                    = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle    = (known after apply)
+   instance_state        = (known after apply)
+   instance_type         = "t3.micro"
+   ipv6_address_count    = (known after apply)
+   ipv6_addresses       = (known after apply)
+   key_name              = (known after apply)
+   monitoring            = (known after apply)
+   outpost_arn           = (known after apply)
+   password_data         = (known after apply)
+   placement_group       = (known after apply)
+   placement_group_id    = (known after apply)
+   placement_partition_number = (known after apply)
+   primary_network_interface_id = (known after apply)
+   private_dns           = (known after apply)
+   private_ip            = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

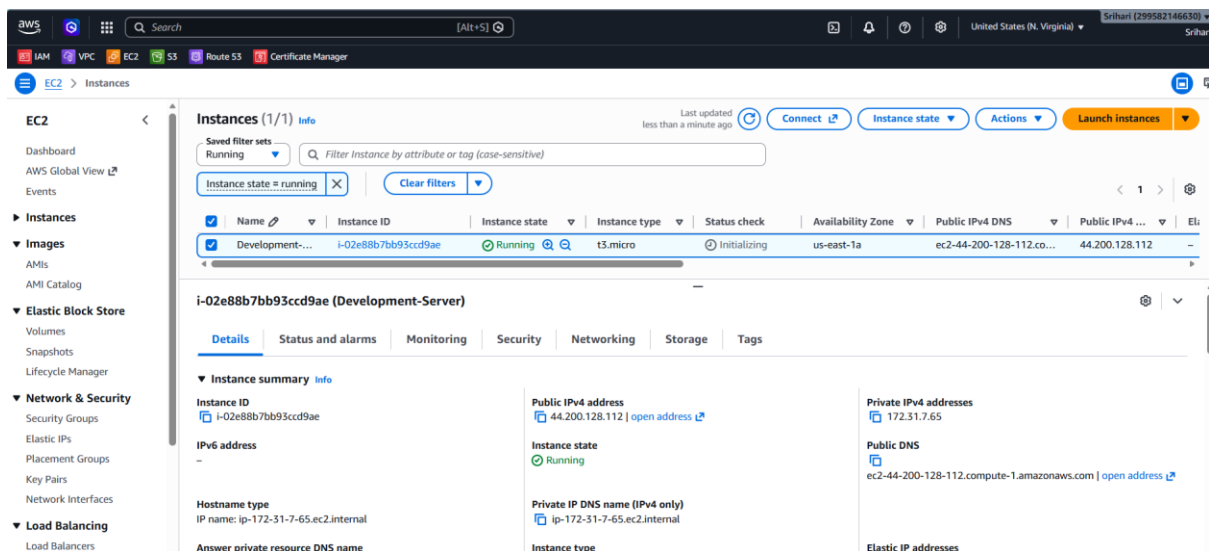
Enter a value: yes

module.development.aws_instance.my_instance: Creating...
module.development.aws_instance.my_instance: Still creating... [00m10s elapsed]
module.development.aws_instance.my_instance: Creation complete after 17s [id=i-02e88b7bb93ccd9ae]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT\development>

```

- Open **AWS Console**
- Go to **EC2 → Instances**
- Check your instance state is **Running**
- Verify **AMI ID** and **Instance Type (t3.micro)**



- In terminal (project root), create new workspace:

terraform workspace new projectA

- List workspaces:

terraform workspace list

- Switch workspace:

terraform workspace select default

terraform workspace select projectA

3). Provision EC2, S3, and VPC using Terraform modules.

&

4). Provision EC2 for 3 different environments (Dev, Staging, and Prod) using Terraform workspaces.

terraform modules project

terraform-modules-project/

|

└─ provider.tf

└─ variables.tf

└─ main.tf

└─ outputs.tf

|

└─ modules/

|

└─ vpc/

| └─ main.tf

| └─ variables.tf

| └─ outputs.tf

|

└─ s3/

| └─ main.tf

| └─ variables.tf

| └─ outputs.tf

|

└─ ec2/

└─ main.tf

└─ variables.tf

└─ outputs.tf

Step 1: i write this scripts of main.tf of Ec2,vpc,s3

#this is a script of ec2/main.tf

```
resource "aws_instance" "this" {

  ami          = var.ami_id

  instance_type = var.instance_type

  subnet_id    = var.subnet_id

  key_name      = var.key_name


  root_block_device {

    volume_size      = var.volume_size

    volume_type       = "gp3"

    encrypted         = true

    delete_on_termination = true

  }

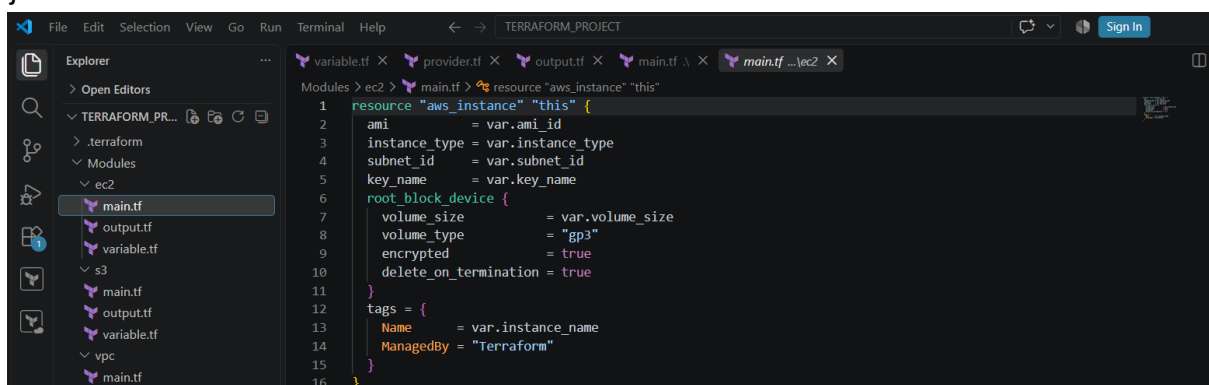

  tags = {

    Name      = var.instance_name

    ManagedBy = "Terraform"

  }

}
```



#this is a script of vpc/main.tf

Create the VPC

```
resource "aws_vpc" "this" {  
  cidr_block      = var.vpc_cidr  
  enable_dns_hostnames = true  
  enable_dns_support = true
```

```
  tags = {  
    Name = var.vpc_name  
  }  
}
```

Create Public Subnets

```
resource "aws_subnet" "public" {  
  count          = length(var.public_subnet_cidrs)  
  vpc_id         = aws_vpc.this.id  
  cidr_block     = var.public_subnet_cidrs[count.index]  
  availability_zone = var.availability_zones[count.index]  
  map_public_ip_on_launch = true
```

```
  tags = {  
    Name = "${var.vpc_name}-public-${count.index + 1}"  
  }  
}
```

Create Internet Gateway

```
resource "aws_internet_gateway" "igw" {  
  vpc_id = aws_vpc.this.id  
  
  tags = {  
    Name = "${var.vpc_name}-igw"  
  }  
}
```

Create Route Table for Public Subnets

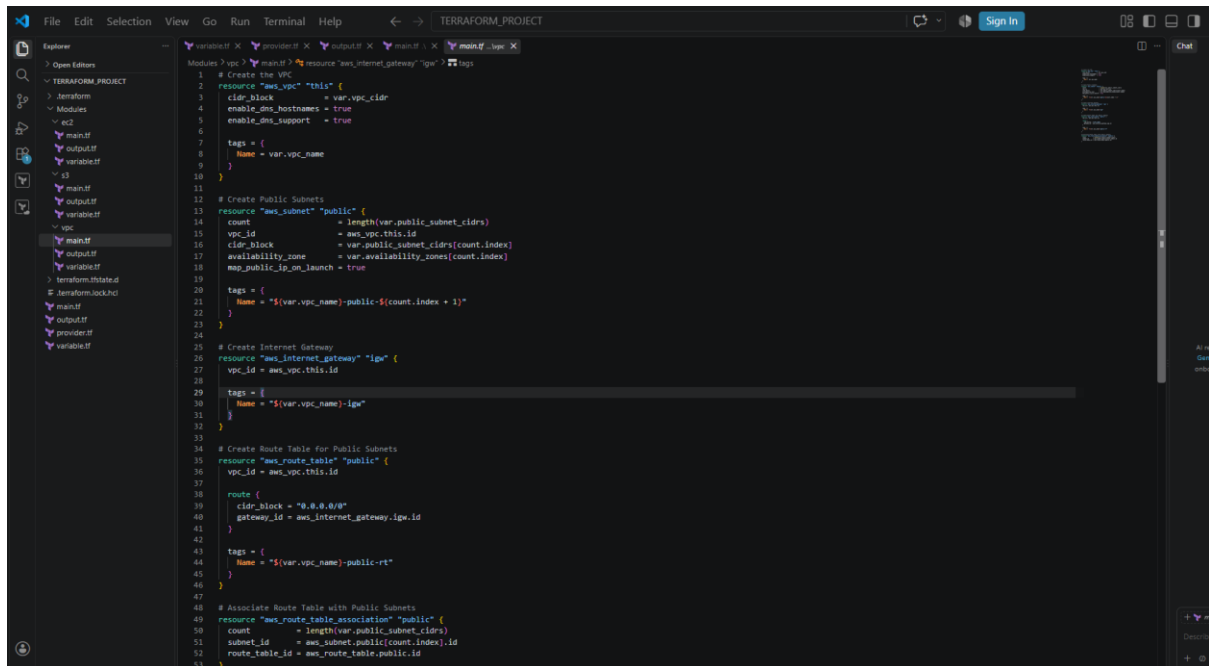
```
resource "aws_route_table" "public" {  
  vpc_id = aws_vpc.this.id  
  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.igw.id  
  }  
  
  tags = {  
    Name = "${var.vpc_name}-public-rt"  
  }  
}
```

Associate Route Table with Public Subnets

```
resource "aws_route_table_association" "public" {  
  count      = length(var.public_subnet_cidrs)  
  subnet_id = aws_subnet.public[count.index].id
```

```
route_table_id = aws_route_table.public.id
```

```
}
```



#this is a script of s3/main.tf

Create the base S3 bucket

```
resource "aws_s3_bucket" "this" {
```

```
    bucket = var.bucket_name
```

```
    force_destroy = var.environment == "dev" ? true : false # Protects prod data
                    from accidental terraform destroy
```

```
    tags = {
```

```
        Name = var.bucket_name
```

```
        Environment = var.environment
```

```
        ManagedBy = "Terraform"
```

```
    }
```

```
}
```

Configure Bucket Versioning

```
resource "aws_s3_bucket_versioning" "versioning" {  
    bucket = aws_s3_bucket.this.id  
    versioning_configuration {  
        status = var.versioning_status  
    }  
}
```

Configure Default Server-Side Encryption (using managed S3 key)

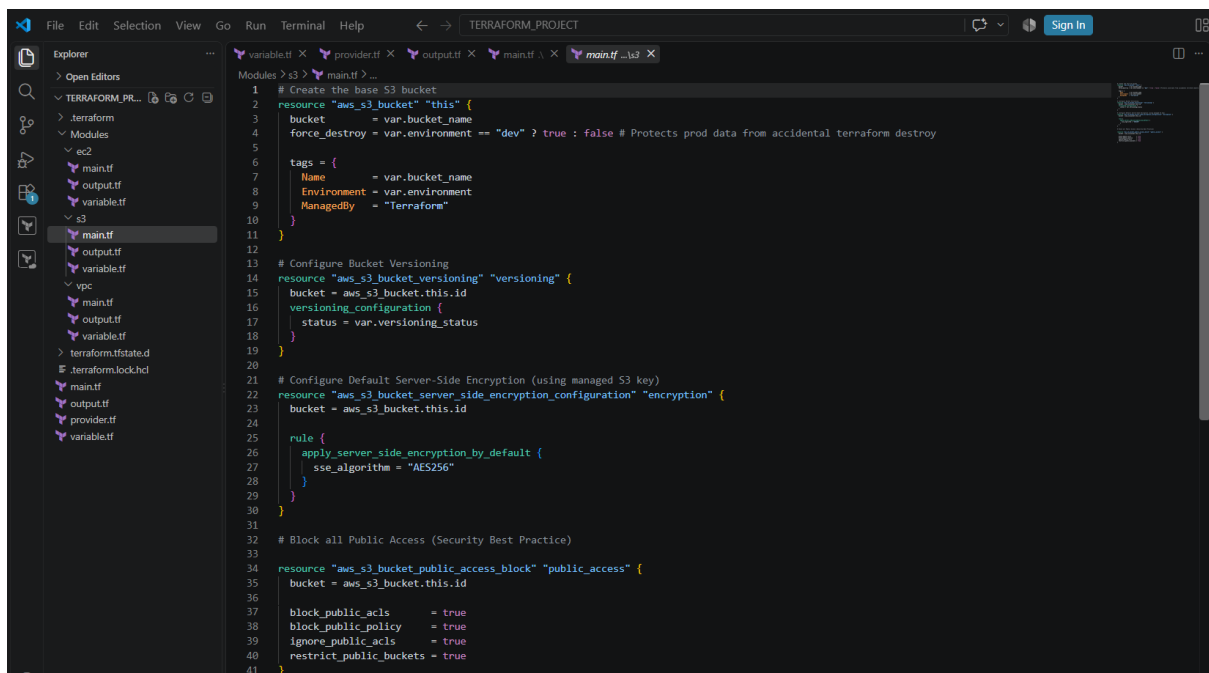
```
resource "aws_s3_bucket_server_side_encryption_configuration" "encryption"  
{  
    bucket = aws_s3_bucket.this.id  
  
    rule {  
        apply_server_side_encryption_by_default {  
            sse_algorithm = "AES256"  
        }  
    }  
}
```

Block all Public Access (Security Best Practice)

```
resource "aws_s3_bucket_public_access_block" "public_access" {  
    bucket = aws_s3_bucket.this.id  
  
    block_public_acls       = true  
    block_public_policy     = true  
    ignore_public_acls     = true
```

```
restrict_public_buckets = true
```

```
}
```



step 2: i write this scripts of variable.tf of Ec2,vpc,s3

#this is a script of ec2/variable.tf

```
variable "instance_name" {
```

```
  type    = string
```

```
  description = "The Name tag for the EC2 instance"
```

```
}
```

```
variable "instance_type" {
```

```
  type    = string
```

```
  description = "The size of the instance"
```

```
  default    = "t3.micro"
```

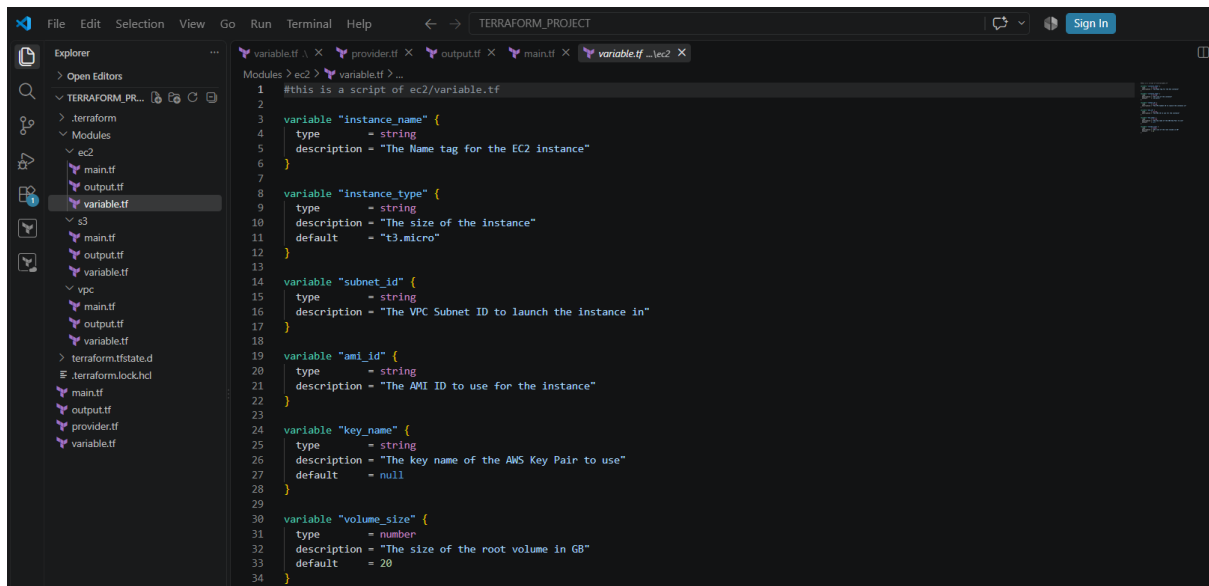
```
}
```

```
variable "subnet_id" {  
    type      = string  
    description = "The VPC Subnet ID to launch the instance in"  
}
```

```
variable "ami_id" {  
    type      = string  
    description = "The AMI ID to use for the instance"  
}
```

```
variable "key_name" {  
    type      = string  
    description = "The key name of the AWS Key Pair to use"  
    default    = null  
}
```

```
variable "volume_size" {  
    type      = number  
    description = "The size of the root volume in GB"  
    default    = 20  
}
```



#this is a script of vpc/variable.tf

```
variable "vpc_cidr" {  
  
  type    = string  
  
  description = "The CIDR block for the VPC"  
  
}
```

```
variable "vpc_name" {  
  
  type    = string  
  
  description = "Name tag for the VPC"  
  
}
```

```
variable "public_subnet_cidrs" {  
  
  type    = list(string)  
  
  description = "CIDR blocks for the public subnets"  
  
}
```

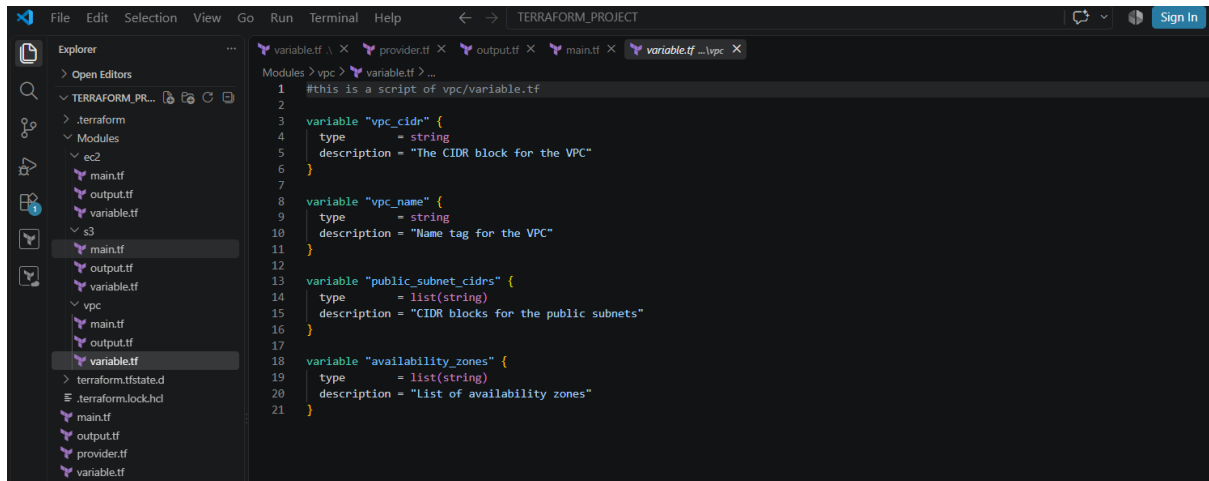
```
variable "availability_zones" {
```

```

type      = list(string)

description = "List of availability zones"
}

```



#this is a script of s3/variable.tf

```

variable "bucket_name" {

  type      = string

  description = "The globally unique name of the S3 bucket"

}

```

```

variable "environment" {

  type      = string

  description = "Deployment environment tag (e.g., dev, prod)"

  default    = "dev"

}

```

```

variable "versioning_status" {

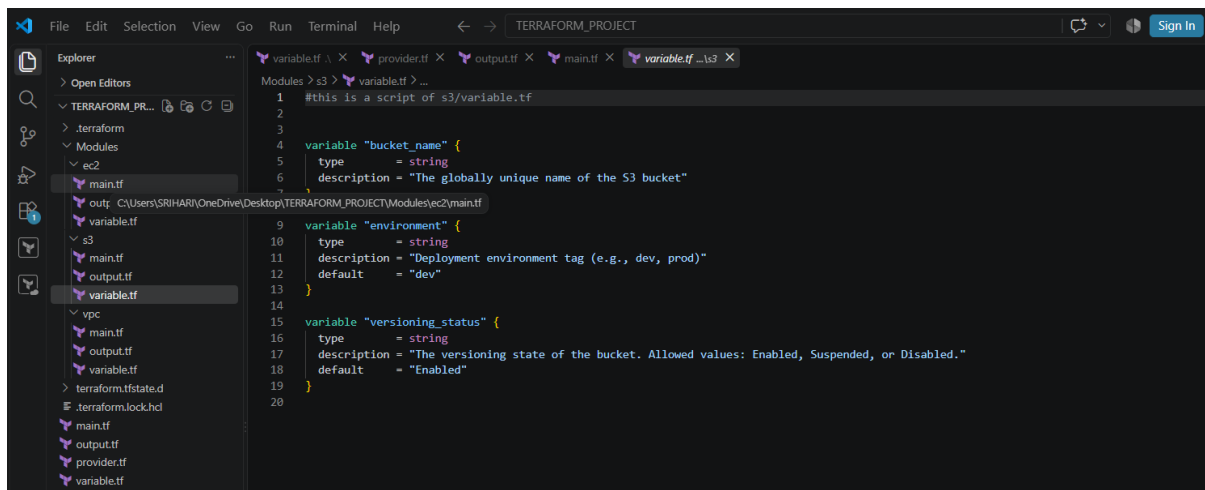
  type      = string

```

```
description = "The versioning state of the bucket. Allowed values: Enabled,
Suspended, or Disabled."
```

```
default    = "Enabled"
```

```
}
```



step 3: i write this scripts of output.tf of Ec2,vpc,s3

#this is a script of ec2/output.tf

```
output "instance_id" {
  value    = aws_instance.this.id
  description = "The ID of the EC2 instance"
}
```

```
output "public_ip" {
  value    = aws_instance.this.public_ip
  description = "The public IP address assigned to the instance"
}
```

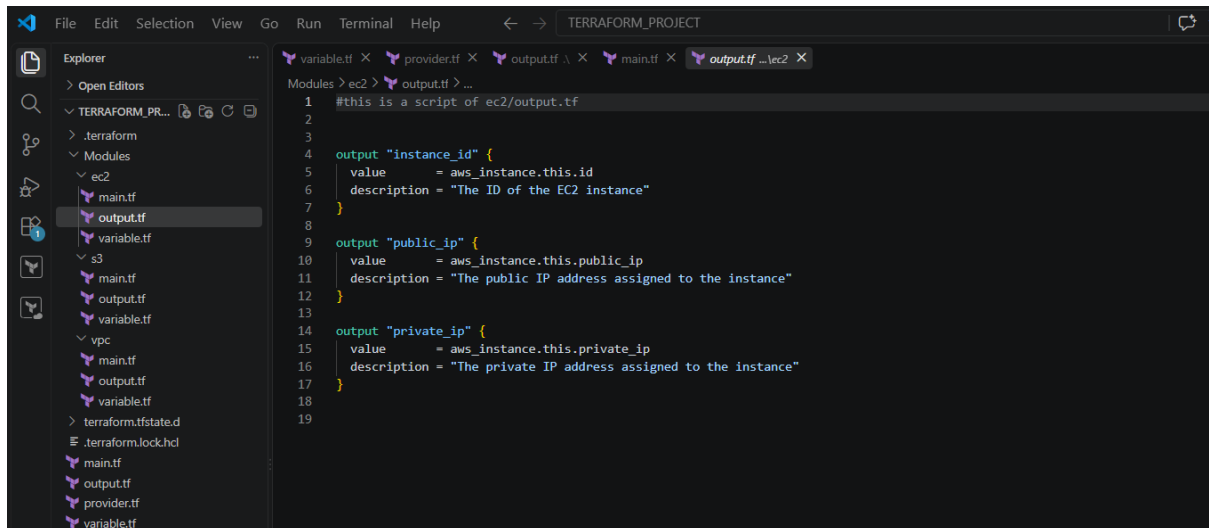
```
output "private_ip" {
```

```

value      = aws_instance.this.private_ip

description = "The private IP address assigned to the instance"
}

```



#this is a script of vpc/output.tf

```

output "vpc_id" {

    value      = aws_vpc.this.id

    description = "The ID of the created VPC"
}

```

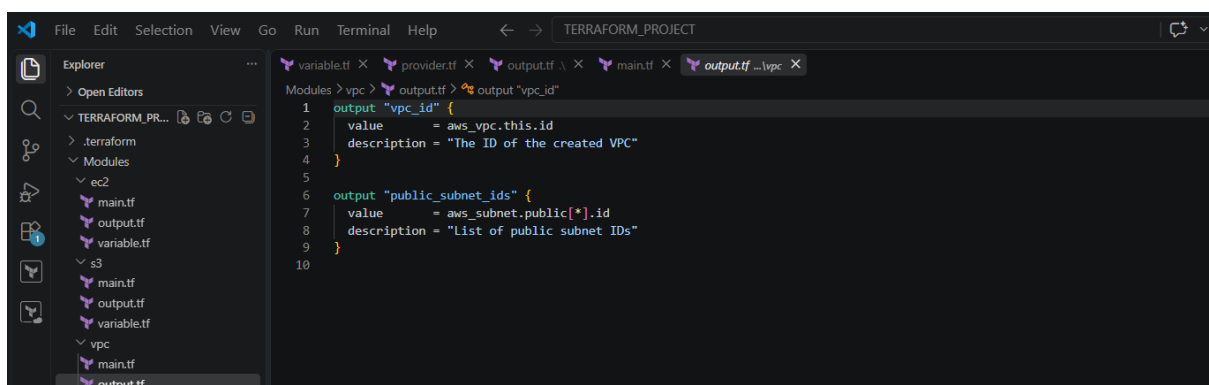
```

output "public_subnet_ids" {

    value      = aws_subnet.public[*].id

    description = "List of public subnet IDs"
}

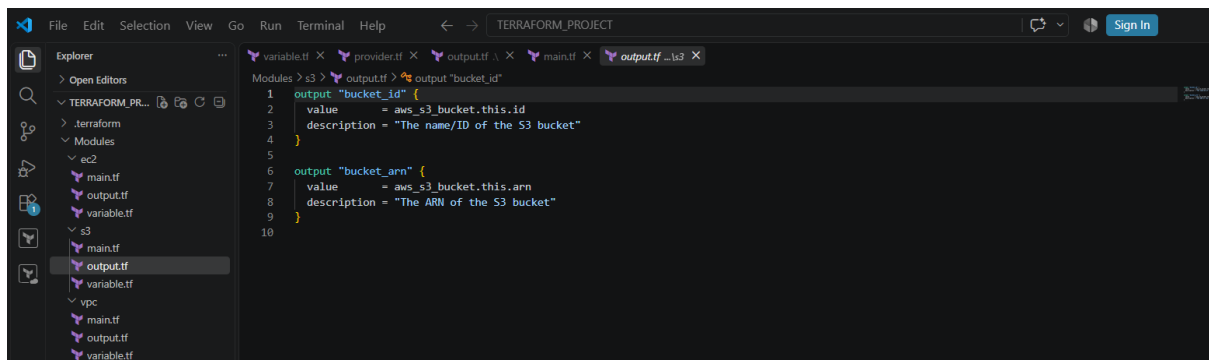
```



#this is a script of s3/output.tf

```
output "bucket_id" {  
    value      = aws_s3_bucket.this.id  
    description = "The name/ID of the S3 bucket"  
}
```

```
output "bucket_arn" {  
    value      = aws_s3_bucket.this.arn  
    description = "The ARN of the S3 bucket"  
}
```



Now this script are write for Root files

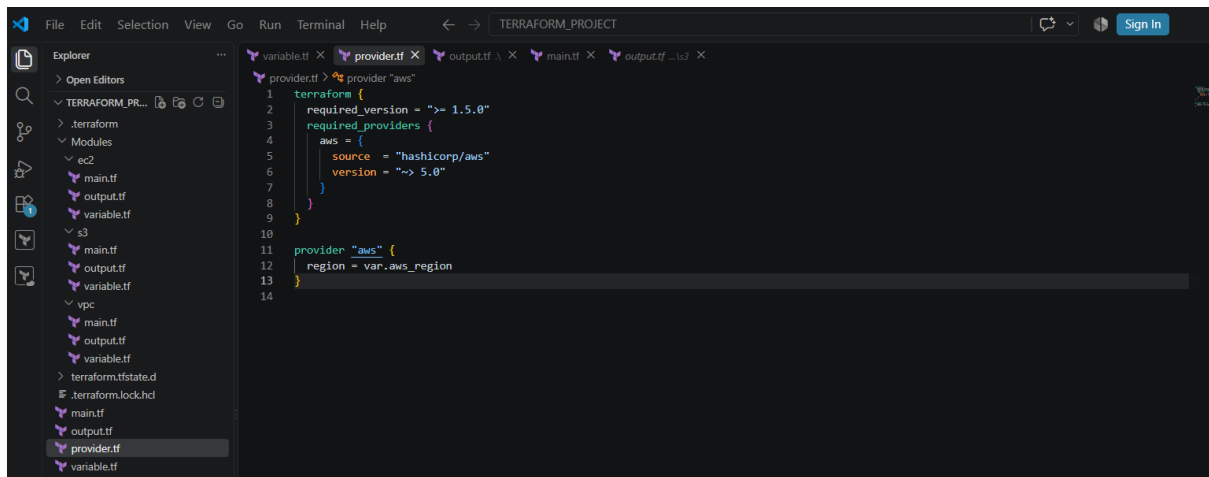
terraform-modules-project/

```
|  
├─ provider.tf  
├─ variables.tf  
├─ main.tf  
└─ outputs.tf
```

This below Script for provider.tf

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = var.aws_region  
}
```



This below script for variable.tf

```
variable "aws_region" {  
    type    = string  
    default = "us-east-1"  
}
```

```
variable "ami_id" {  
    type      = string  
    description = "The AMI ID to use for the EC2 instance"  
    default    = "ami-0354c98ae10b02961" # Replace with your target regional  
Linux AMI  
}
```

```
locals {  
    # Automatically detects the active workspace (dev, staging, or prod)  
    env = terraform.workspace
```

Environment-specific configuration maps

```
instance_type = {  
    dev    = "t3.micro"  
    staging = "t3.small"  
    prod   = "c7i-flex.large"  
}
```

```
vpc_cidr = {  
    dev    = "10.0.0.0/16"  
    staging = "10.1.0.0/16"
```

```
    prod    = "10.2.0.0/16"
  }
```

```
public_subnets = {
  dev    = ["10.0.1.0/24", "10.0.2.0/24"]
  staging = ["10.1.1.0/24", "10.1.2.0/24"]
  prod   = ["10.2.1.0/24", "10.2.2.0/24"]
}
```

```
availability_zones = ["us-east-1a", "us-east-1b"]
}
```

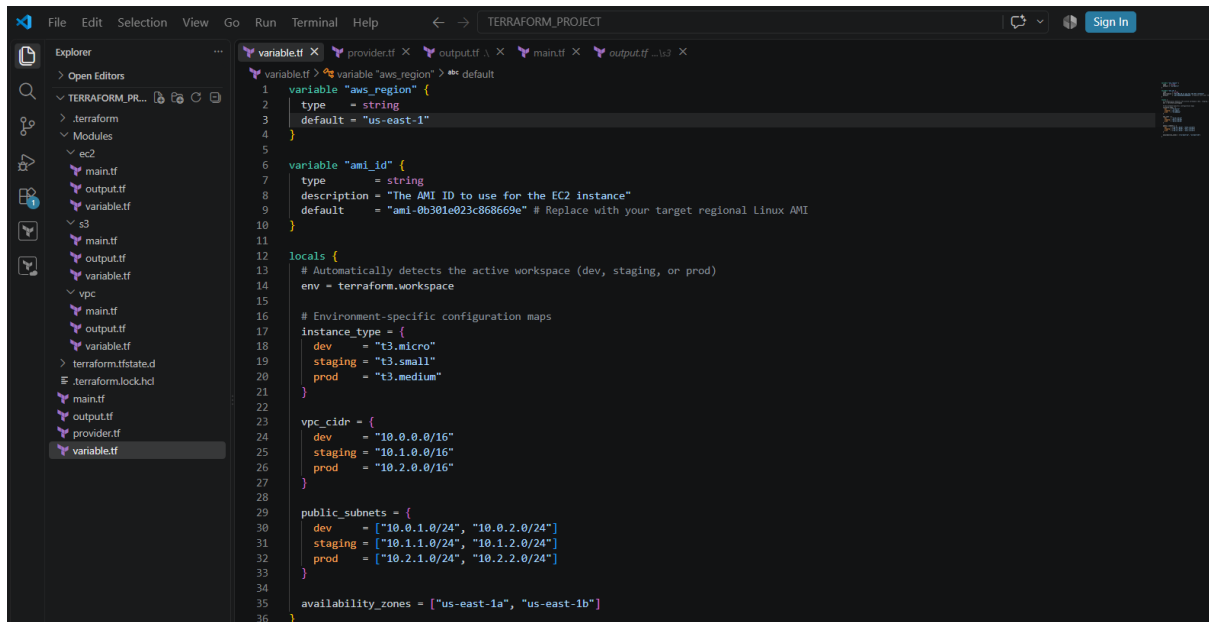
This script below for output.tf

```
output "current_environment" {
  value    = local.env
  description = "The active workspace environment."
}
```

```
output "vpc_id" {
  value = module.vpc.vpc_id
}
```

```
output "ec2_public_ip" {
  value = module.ec2.public_ip
}
```

```
output "s3_bucket_name" {
    value = module.s3.bucket_id
}
```



This below script for main.tf

1. Provision the VPC Module

```
module "vpc" {
    source = "../Modules/vpc"

    vpc_name      = "${local.env}-vpc"
    vpc_cidr      = local.vpc_cidr[local.env]
    public_subnet_cidrs = local.public_subnets[local.env]
    availability_zones = local.availability_zones
}
```

2. Provision the S3 Module

```
module "s3" {
```

```
source = "./Modules/s3"
```

```
# S3 bucket names must be globally unique; appending local.env avoids collisions
```

```
bucket_name    = "my-unique-app-storage-${local.env}-bucket"
```

```
environment    = local.env
```

```
versioning_status = local.env == "prod" ? "Enabled" : "Suspended"
```

```
}
```

```
# 3. Provision the EC2 Module
```

```
module "ec2" {
```

```
source = "./Modules/ec2"
```

```
instance_name = "${local.env}-web-server"
```

```
instance_type = local.instance_type[local.env]
```

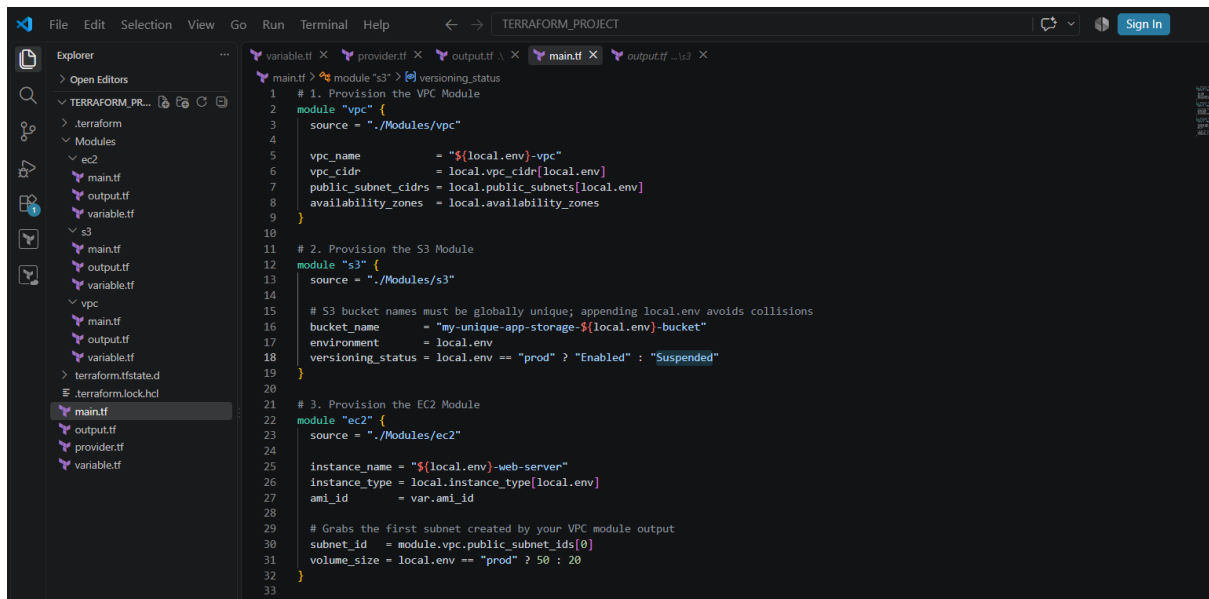
```
ami_id        = var.ami_id
```

```
# Grabs the first subnet created by your VPC module output
```

```
subnet_id     = module.vpc.public_subnet_ids[0]
```

```
volume_size   = local.env == "prod" ? 50 : 20
```

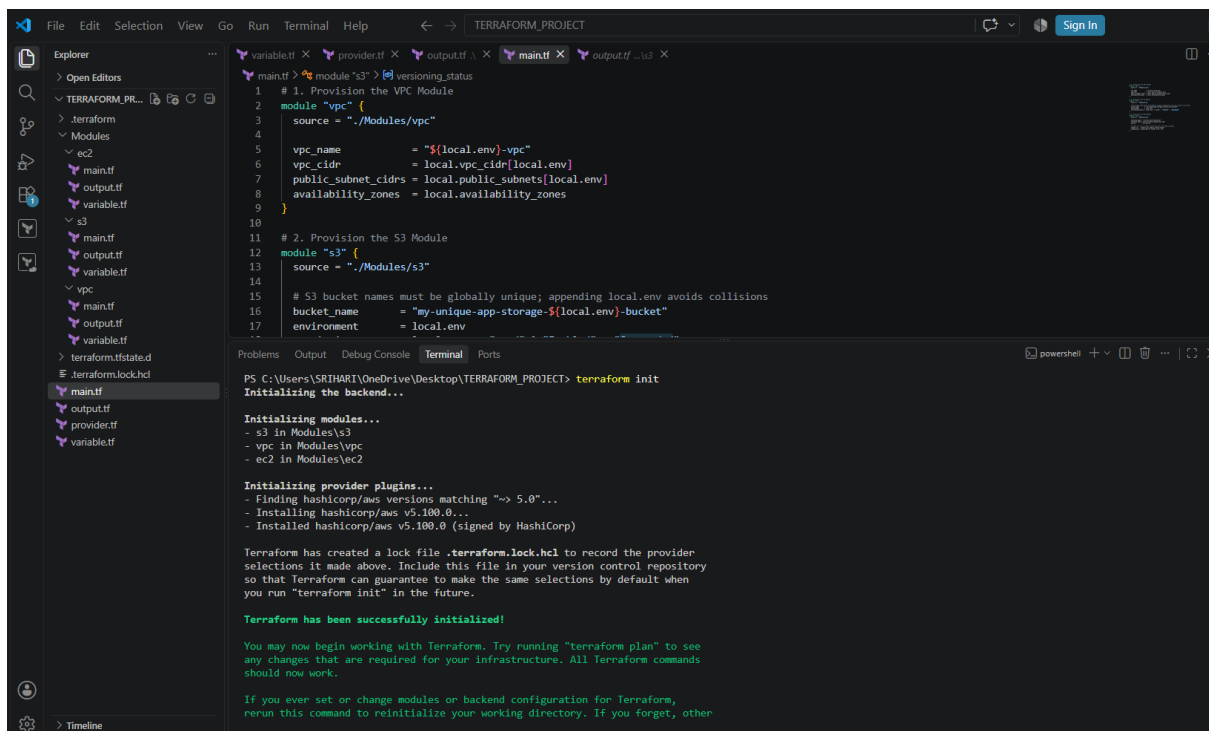
```
}
```



Final step: Command for multi environment

Step 1: Initialize the configuration and modules

terraform init



Step 2: Deploy Dev

terraform workspace new dev

terraform workspace select dev

terraform apply -auto-approve

```
Problems Output Debug Console Terminal Ports
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace list
* default

PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace new dev
Created and switched to workspace "dev"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace select dev
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

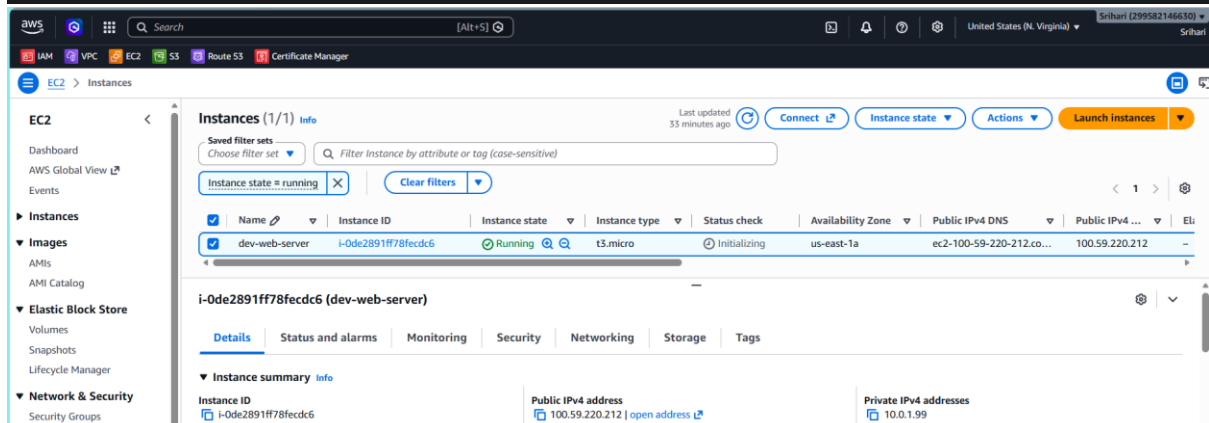
Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
+ resource "aws_instance" "this" {
  + ami              = "ami-0b301e023c868669e"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count    = (known after apply)
  + cpu_threads_per_core = (known after apply)
}
```

```
Problems Output Debug Console Terminal Ports
module.vpc.aws_internet_gateway.igw: Creating...
module.vpc.aws_internet_gateway.igw: Creation complete after 2s [id=igw-01b61c06db67ee0f0]
module.vpc.aws_route_table.public: Creating...
module.vpc.aws_route_table.public: Creation complete after 3s [id=rtb-064171c77bc203b37]
module.vpc.aws_subnet.public[0]: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public[1]: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public[0]: Creation complete after 12s [id=subnet-0389b4608bcb00019]
module.vpc.aws_subnet.public[1]: Creation complete after 13s [id=subnet-0cb2ca864c6e40ddf]
module.vpc.aws_route_table_association.public[1]: Creating...
module.ec2.aws_instance.this: Creating...
module.vpc.aws_route_table_association.public[0]: Creating...
module.vpc.aws_route_table_association.public[1]: Creation complete after 1s [id=rtbassoc-0642fe32b76848fb1]
module.vpc.aws_route_table_association.public[0]: Creation complete after 1s [id=rtbassoc-0cc98b6f1ffffbc05]
module.ec2.aws_instance.this: Still creating... [00m10s elapsed]
module.ec2.aws_instance.this: Creation complete after 15s [id=i-0de2891ff78fedc6]

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:
current_environment = "dev"
ec2_public_ip = "100.59.220.212"
s3_bucket_name = "my-unique-app-storage-dev-bucket"
vpc_id = "vpc-019c7f6e51f7e75a2"
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT>
```

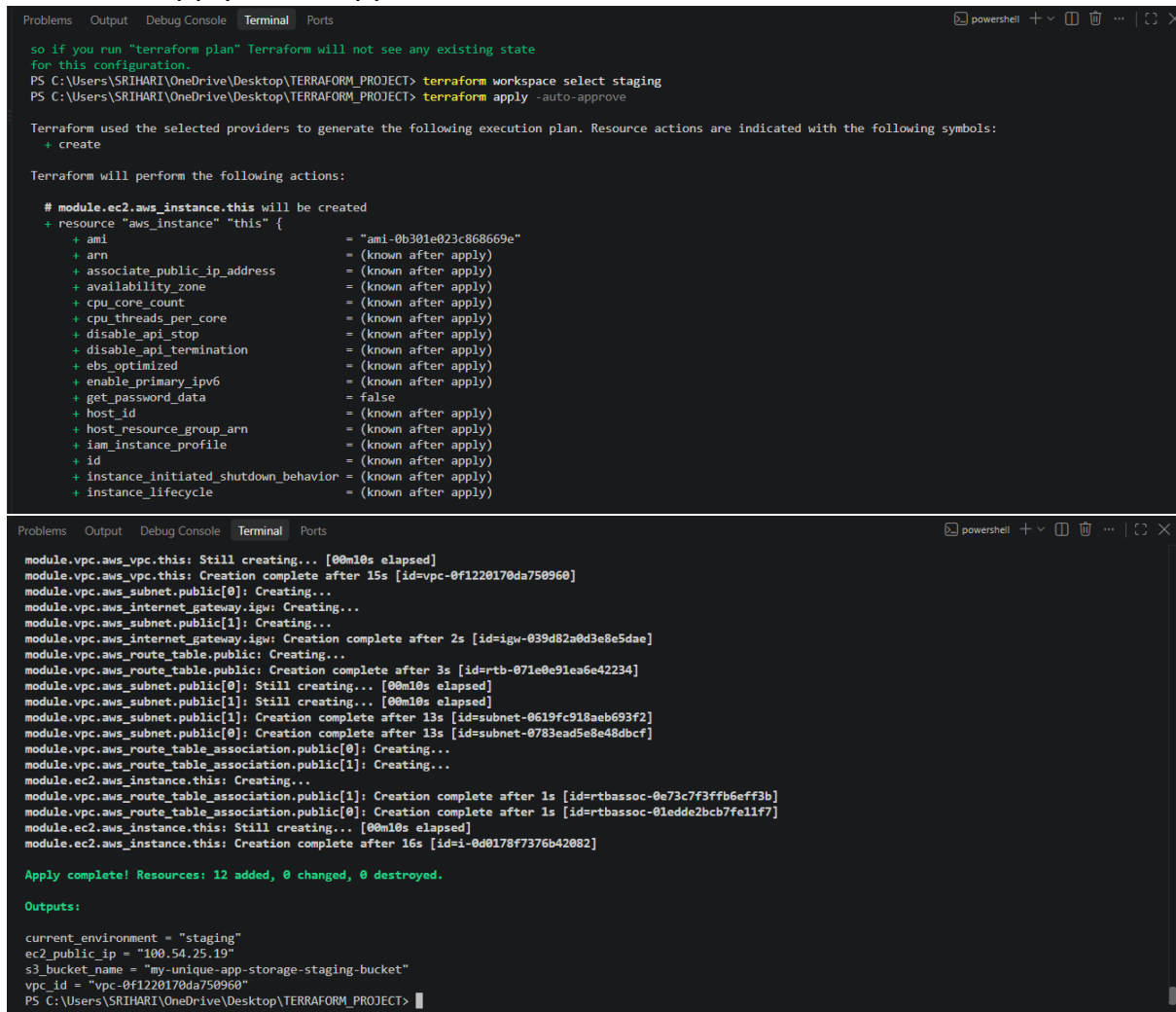


Step 3: Deploy Staging

terraform workspace new staging

terraform workspace select staging

terraform apply -auto-approve



```
Problems Output Debug Console Terminal Ports
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace select staging
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
+ resource "aws_instance" "this" {
  + ami                  = "ami-0b301e023c868669e"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
  + cpu_threads_per_core  = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile  = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle    = (known after apply)

module.vpc.aws_vpc.this: Still creating... [00m10s elapsed]
module.vpc.aws_vpc.this: Creation complete after 15s [id=vpc-0f1220170da750960]
module.vpc.aws_subnet.public[0]: Creating...
module.vpc.aws_subnet.public[1]: Creating...
module.vpc.aws_internet_gateway.igw: Creating...
module.vpc.aws_subnet.public[1]: Creation complete after 2s [id=igw-039d82a0d3e8e5dae]
module.vpc.aws_subnet.public[0]: Creation complete after 3s [id=rtb-071e0e91ea6e42234]
module.vpc.aws_subnet.public[0]: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public[1]: Still creating... [00m10s elapsed]
module.vpc.aws_subnet.public[1]: Creation complete after 13s [id=subnet-0619fc918aeb693f2]
module.vpc.aws_subnet.public[0]: Creation complete after 13s [id=subnet-0783ead5e8e48dbcf]
module.vpc.aws_route_table_association.public[0]: Creating...
module.vpc.aws_route_table_association.public[1]: Creating...
module.vpc.aws_instance.this: Creating...
module.vpc.aws_route_table_association.public[1]: Creation complete after 1s [id=rtbassoc-0e73c7f3ffb6eff3b]
module.vpc.aws_route_table_association.public[0]: Creation complete after 1s [id=rtbassoc-01edde2bcb7fe11f7]
module.ec2.aws_instance.this: Still creating... [00m10s elapsed]
module.ec2.aws_instance.this: Creation complete after 16s [id=i-0d0178f7376b42082]

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:
current_environment = "staging"
ec2_public_ip      = "100.54.25.19"
s3_bucket_name     = "my-unique-app-storage-staging-bucket"
vpc_id             = "vpc-0f1220170da750960"
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT>
```

Step 4: Deploy Prod

terraform workspace new prod

terraform workspace select prod

terraform apply -auto-approve

```
Problems Output Debug Console Terminal Ports powershell
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace new prod
Created and switched to workspace "prod"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform workspace select prod
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT> terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# module.ec2.aws_instance.this will be created
+ resource "aws_instance" "this" {
  + ami              = "ami-0b301e023c868669e"
  + arn              = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone = (known after apply)
  + cpu_core_count   = (known after apply)
  + cpu_threads_per_core = (known after apply)
  + disable_api_stop  = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized     = (known after apply)
  + enable_primary_ipv6 = (known after apply)
  + get_password_data = false
  + host_id           = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile = (known after apply)

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)
module.ec2.aws_instance.this: Creating...
module.ec2.aws_instance.this: Still creating... [00m10s elapsed]
module.ec2.aws_instance.this: Creation complete after 17s [id=i-0e116624d5471f889]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

current_environment = "prod"
ec2_public_ip = "3.219.240.238"
s3_bucket_name = "my-unique-app-storage-prod-bucket"
vpc_id = "vpc-0db4bc5526ed6a352"
PS C:\Users\SRIHARI\OneDrive\Desktop\TERRAFORM_PROJECT>
```

EC2 > Instances

EC2

Dashboard

AWS Global View

Events

Instances

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

1 Metrics Monitor

Instances (3) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Filter Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4	ELB
☐	dev-web-server	i-0de2891f78fecd6	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-100-59-220-212.co...	100.59.220.212	-
☐	prod-web-server	i-0e116624d5471f889	Running	t3.micro	Initializing	us-east-1a	ec2-3-219-240-238.co...	3.219.240.238	-
☐	staging-web-s...	i-0d0178f7376b42082	Running	t3.small	3/3 checks passed	us-east-1a	ec2-100-54-25-19.com...	100.54.25.19	-

All instances

VPC > Your VPCs

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Your VPCs (1/4) Info

Last updated less than a minute ago

Actions

Create VPC

Find VPCs by attribute or tag

☒	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
☒	-	vpc-00148488443198382	Available	-	-	Off	172.31.0.0/16
☐	dev-vpc	vpc-019c7f6e51fbc75a2	Available	-	-	Off	10.0.0.0/16
☐	staging-vpc	vpc-0f1220170da750960	Available	-	-	Off	10.1.0.0/16
☐	prod-vpc	vpc-0db4bc5526ed6a352	Available	-	-	Off	10.2.0.0/16

Amazon S3

Buckets

General purpose buckets

Directory buckets

General purpose buckets (4) Info

Buckets are containers for data stored in S3.

Find buckets by name

☐	Name	AWS Region	Creation date
☐	my-unique-app-storage-dev-bucket	US East (N. Virginia) us-east-1	September 15, 2026, 16:57:56 (UTC+05:30)
☐	my-unique-app-storage-prod-bucket	US East (N. Virginia) us-east-1	September 15, 2026, 17:58:25 (UTC+05:30)
☐	my-unique-app-storage-staging-bucket	US East (N. Virginia) us-east-1	September 15, 2026, 17:54:27 (UTC+05:30)

Account snapshot

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

View dashboard

External access summary

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.